

11

FEATURE

FEATURE 11 OF 17 - GETTING THE DATA OUT

NetCDF Export & HTTP API

The feature that turns a demo into infrastructure - standard files and a live programmatic interface other systems can call.

PORT	SERVICE	FORMATS	ROUTES
8511	phase2.api.app (uvicorn)	NetCDF + JSON	4, with live docs

Before this existed there was no way to get a reconstruction out of the system at all - verified by searching the codebase and finding zero web framework, zero download button. A dashboard nobody can extract data from is a demonstration. This is what makes it a service.

1. What this feature is, in plain words

Two ways out. **NetCDF** is the standard scientific data format for gridded ocean and atmosphere fields - every serious ocean tool reads it, so exporting to NetCDF means our output drops straight into an existing workflow. **The HTTP API** lets another program ask for a profile or a whole field over the network, with automatically generated documentation any developer can read.

Getting a file format right is mostly about getting absences right, and three decisions here are worth stating to a jury because each one prevents a specific false claim. Missing values are marked explicitly, because left to a reader's default a land cell becomes a **0 degrees measurement**. A temperature-only file **omits** the salinity variables entirely rather than writing a grid full of blanks, because a blank grid asserts 'this file has salinity, missing everywhere' - a different and false claim. And masks are stored as integer flags with their meanings attached, because NetCDF has no boolean type and a silent conversion to float is how a mask stops being a mask.

Two implementation choices in the API are load-bearing rather than stylistic, and both are enforced by tests that parse the source code. The request handlers are deliberately synchronous, because an asynchronous handler calling the predictor would freeze the entire service for over thirty seconds. And every prediction runs inside a lock, because the predictor is shared and two simultaneous requests would otherwise return **plausible wrong answers** rather than raising an error.

THE EVERYDAY PICTURE

A weather app that shows you a forecast is useful. A weather API that other apps can call is infrastructure. This is the step from one to the other.

2. What you actually see on the screen

Four routes, all verified live:

- **/health** - is the service up, and which checkpoint is it serving.
- **/coverage** - which dates the model can actually answer for.
- **/profile** - the 15-depth profile at a point and date, as JSON, in about 1.1 seconds.
- **/field.nc** - the whole basin for one date as a NetCDF file, 3.29 MB in about 34 seconds.
- **/docs** - interactive, automatically generated documentation for all four routes.
- Every response carries a ten-key provenance block, including the checkpoint's hash and the code commit - and a test asserts that contract holds rather than trusting it.

3. How to run it, step by step

Start the service, then call it. It binds to localhost only.

RUN THIS

```
PYTHONPATH=src python -m uvicorn phase2.api.app:app --host 127.0.0.1 --port 8511 --workers 1
```

- 1 Run the command above. The service starts on **http://127.0.0.1:8511**.
- 2 Open **http://127.0.0.1:8511/docs** in a browser. All four routes render with a Try It button - this is the most impressive thirty seconds of this feature.
- 3 Call **/coverage** first to see the valid date range.
- 4 Call **/profile** with a latitude, longitude and date. Point out that the response body contains no null tokens and carries the checkpoint hash.
- 5 Ask for a date **outside** the range on purpose. It returns a 422 error that names the valid range - it refuses rather than quietly snapping to the nearest date.
- 6 Call **/field.nc** to download a real NetCDF file, then reopen it to show it carries `input_source = satellite`.

EXPORT ONE DAY TO NETCDF WITHOUT THE SERVICE

```
PYTHONPATH=src python scripts/phase2/export_field.py --date 2026-06-23 --out field.nc
```

4. Why the problem statement needs this

A national agency cannot consume a Streamlit page. It consumes files and endpoints. This feature is what makes the output usable by IMD, by a forecast model, by a downstream application, or by a researcher who wants to check our work in their own tools.

It is also the feature that makes reproducibility real: every exported file carries the checkpoint hash and code commit that produced it, so a number quoted from one of our files can always be traced back to the exact model that generated it.

PS requirement	How this feature answers it
Deliver a usable product	Standard NetCDF plus a documented HTTP interface - both formats a sponsor can actually integrate.
Reproducibility and provenance	Ten provenance keys on every response, asserted by a test rather than assumed.
Satellite-input compliance	The exported file carries <code>input_source = satellite</code> and the checkpoint hash, so the claim travels with the data.

5. The measured numbers behind it

Measured live, on this machine, rather than estimated [VERIFIED]:

Operation	Measured cost	Note
/profile	about 1.1 s, 15 depths	No null tokens in the body.
/field.nc	3.29 MB in 34.3 s	Reopens as valid NetCDF carrying input_source = satellite and the checkpoint hash.
Whole-field prediction	32.09 s median of 4 runs, 11,832 ocean cells, batch 512	This replaced an estimate that said 'seconds on GPU, under a minute on CPU' - an estimate standing where a reader takes a cost figure.
Predictor load	7.20 s, paid once	Total warm 32.84 s, total cold 39.90 s.
Out-of-range date	HTTP 422, naming the valid range	It refuses rather than snapping to the nearest available date.

- **The measurement found a real bug.** Timing the export revealed that a GPU was available and the inference path had never used it - the predictor loaded onto the CPU and was never moved. After the fix: **CPU 37.57 s against CUDA 8.86 s, a 4.24x speedup**, with a maximum difference of 6.9e-04 degC across 153,291 cells - float noise against a 0.9006 degC headline.
- The export is deliberately kept on the CPU anyway, so that a downloaded file cannot differ in its last digits from the page displayed beside it.

6. The impact - and who it is for

Who	What changes for them
INCOIS / MoES	A path from our output into their existing systems, in the format their tools already read.
Forecast modellers	Programmatic access to subsurface fields as an input to their own models.
Independent reviewers	The ability to download our actual output and check it in their own software rather than trusting our screenshots.
Downstream developers	Self-documenting endpoints, so building on this needs no conversation with us.

WHY SOCIETY GAINS FROM THIS

Open, standard-format, provenance-carrying data is what lets research outside the original team happen at all. A state fisheries department, a university group or a startup can build on this without permission and without re-implementing anything - and can trace any number they publish back to the exact model that produced it.

7. Say this to the jury

Before this, there was no way to get data out of the system - we checked, and there was no web framework and no download button anywhere in the codebase. Now there is a NetCDF export and a documented HTTP API. Ask it for a date outside our range and it returns an error naming the valid range, rather than quietly giving you the nearest date - which would have been a wrong answer that looked right. And every file carries the checkpoint hash and the code commit that made it, so any number taken from our output can be traced back to the exact model that produced it.

8. Questions the jury will ask

Q. Why is a whole-field export 34 seconds? That seems slow.

A. It is 11,832 ocean cells, each with a full profile, and we measured it rather than estimating it. We also measured a 4.24x GPU speedup and deliberately did not switch the export to GPU, so that a downloaded file cannot differ in its last digits from the page beside it. That is a considered trade, and the measurement is on file.

Q. What stops two simultaneous requests corrupting each other?

A. A lock around every prediction. The predictor is shared process-wide, and without the lock two concurrent requests would return plausible wrong answers instead of raising - which is far more dangerous than a crash. A test using real threads checks this.

Q. Why omit salinity from the file rather than write an empty grid?

A. Because an all-blank salinity grid asserts that the file has salinity and it happens to be missing everywhere. That is a different and false claim. The stage-1 file omits those variables entirely, which correctly says the product does not contain them.

9. The honest limit - say it before they find it

LOCALHOST, SINGLE WORKER, AND NO AUTHENTICATION

This is a demonstration service bound to 127.0.0.1 with one worker and no access control. Operational deployment would need authentication, rate limiting, horizontal scaling and a caching layer in front of the 34-second field endpoint. None of that is built, and we would rather say so than imply the service is production-ready.

IN ONE LINE

Standard NetCDF files and a documented HTTP service, each carrying the checkpoint hash that produced it - the step from demonstration to infrastructure.

3.29 MB

A WHOLE BASIN-DAY AS NETCDF

1.1 s

A SINGLE PROFILE OVER HTTP

HTTP 422

FOR A BAD DATE - REFUSES RATHER THAN SNAPS

Feature 11 of 17. Next: Feature 12.